

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA ĐÀO TẠO SAU ĐẠI HỌC



BÁO CÁO BÀI TẬP NHÓM
MÔN: CÁC HỆ THỐNG PHÂN TÁN
HỆ THỐNG BÁN HÀNG TRỰC TUYẾN SỬ DỤNG
KIẾN TRÚC MICROSERVICES

Giảng viên hướng dẫn : TS. Kim Ngọc Bách

Lớp : M25CQHT01-B

Nhóm 9

Nguyễn Thị Hường : B25CHHT027

Phạm Hồng Nhung : B25CHHT045

Phan Tiến Thành : B25CHHT053

Hà Nội - 2025

MỤC LỤC

I.Đặt vấn đề	3
1.Bài toán đặt ra	3
2.Thách thức	4
II.Phân tích chức năng	4
1. Thiết kế chức năng tổng quan	4
1.2. Quản lý sản phẩm	5
1.3. Quản lý đơn hàng	5
1.4. Xác thực và phân quyền bằng JWT	6
1.5. API Gateway làm entry point duy nhất cho client	6
2. Các tính năng cụ thể	6
2.1.User Service	6
2.2.Product Service	7
2.3.Order Service	7
2.4.API Gateway	7
3.Thiết kế phần mềm	7
3.1.Kiến trúc tổng thể	7
3.2. Database Design	8
3.3. Giao tiếp giữa các service	9
3.4. Authentication Flow	9
4. Demo	9
4.1. User services	9
4.2.Product service	10
4.3.Order Services	13
5. Hướng Dẫn Chạy Demo	15
5.1. Yêu cầu hệ thống	15
5.2. Các bước chạy demo	15
5.3. Thông tin kết nối	16
6. Source Code	16

I. Đặt vấn đề

1. Bài toán đặt ra

Trong bối cảnh thương mại điện tử phát triển mạnh mẽ, các hệ thống bán hàng online cần đáp ứng các yêu cầu ngày càng cao về:

- Khả năng mở rộng (scalability) khi số lượng người dùng và giao dịch tăng nhanh.
- Độ tin cậy (reliability) và tính sẵn sàng cao (high availability).
- Dễ dàng bảo trì, phát triển tính năng mới mà không ảnh hưởng đến toàn hệ thống.
- Đảm bảo an toàn, bảo mật thông tin người dùng và giao dịch.

Bài toán đặt ra:

Thiết kế và xây dựng một hệ thống bán hàng trực tuyến (e-commerce) có thể:

- Quản lý người dùng, sản phẩm, đơn hàng một cách độc lập.
- Đáp ứng tốt khi hệ thống cần mở rộng hoặc thay đổi.
- Dễ dàng tích hợp thêm các dịch vụ mới trong tương lai.

2. Thách thức

- Tách biệt chức năng nhưng vẫn đảm bảo giao tiếp hiệu quả:
 - Làm sao để các module (user, product, order) hoạt động độc lập nhưng vẫn phối hợp nhịp nhàng?
 - Đảm bảo dữ liệu nhất quán khi có nhiều service cùng thao tác.
- Quản lý dữ liệu phân tán:
 - Mỗi service có database/schema riêng, cần thiết kế để tránh trùng lặp, đồng bộ dữ liệu hợp lý.
 - Xử lý các giao dịch liên quan nhiều service (ví dụ: tạo đơn hàng phải kiểm tra tồn kho sản phẩm).
- Xác thực và phân quyền:
 - Đảm bảo chỉ người dùng hợp lệ mới được truy cập các chức năng nhạy cảm.
 - Quản lý token, bảo mật thông tin truyền giữa các service.
- Khả năng mở rộng và chịu tải:
 - Hệ thống phải dễ dàng scale từng service khi lượng truy cập tăng.
 - Tránh bottleneck tại API Gateway hoặc các service lõi.
- Giám sát, logging và xử lý lỗi:
 - Làm sao để phát hiện, xử lý lỗi nhanh chóng khi có sự cố ở một service mà không ảnh hưởng toàn hệ thống?
 - Theo dõi log, trace request qua nhiều service.

- Triển khai và vận hành:
 - Tự động hóa việc build, deploy, quản lý nhiều service cùng lúc (CI/CD, Docker, Docker Compose).
 - Đảm bảo các service luôn sẵn sàng, dễ dàng cập nhật, rollback khi cần thiết.

II. Phân tích chức năng

1. Thiết kế chức năng tổng quan

Hệ thống bán hàng online (e-commerce) với các chức năng cơ bản:

- Quản lý người dùng (đăng ký, đăng nhập, xem thông tin cá nhân)
- Quản lý sản phẩm (thêm, sửa, xóa, xem danh sách, xem chi tiết)
- Quản lý đơn hàng (tạo đơn hàng, xem danh sách đơn hàng, xem chi tiết đơn hàng)
- Xác thực và phân quyền bằng JWT
- API Gateway làm entry point duy nhất cho client

1.1. Quản lý người dùng

- Đăng ký tài khoản (Register):
 - Người dùng nhập email, mật khẩu, tên.
 - Hệ thống kiểm tra email đã tồn tại chưa, nếu chưa thì tạo user mới và lưu vào database.
 - Trả về thông tin user vừa tạo (không bao gồm mật khẩu).
- Đăng nhập (Login):
 - Người dùng nhập email và mật khẩu.
 - Hệ thống kiểm tra thông tin đăng nhập, nếu đúng sẽ sinh ra một JWT token.
 - Trả về JWT token cho client, client dùng token này cho các request cần xác thực.
- Xem thông tin cá nhân (Get Profile):
 - Người dùng gửi request kèm JWT token.
 - Hệ thống xác thực token, lấy userId từ token và trả về thông tin user tương ứng.

1.2. Quản lý sản phẩm

- Thêm sản phẩm mới (Create Product):
 - Admin gửi request với thông tin sản phẩm (tên, mô tả, giá, tồn kho, ảnh...).
 - Hệ thống xác thực quyền admin, lưu sản phẩm mới vào database.
- Sửa sản phẩm (Update Product):

- Admin gửi request cập nhật thông tin sản phẩm theo id.
- Hệ thống xác thực quyền admin, cập nhật thông tin sản phẩm trong database.
- Xóa sản phẩm (Delete Product):
 - Admin gửi request xóa sản phẩm theo id.
 - Hệ thống xác thực quyền admin, xóa sản phẩm khỏi database.
- Xem danh sách sản phẩm (List Products):
 - Người dùng gửi request, có thể kèm phân trang (page, limit).
 - Hệ thống trả về danh sách sản phẩm, hỗ trợ phân trang.
- Xem chi tiết sản phẩm (Get Product Detail):
 - Người dùng gửi request với id sản phẩm.
 - Hệ thống trả về thông tin chi tiết của sản phẩm đó.

1.3. Quản lý đơn hàng

- Tạo đơn hàng mới (Create Order):
 - Người dùng gửi request với danh sách sản phẩm muốn mua (productId, quantity), kèm JWT token.
 - Hệ thống xác thực token, kiểm tra tồn kho từng sản phẩm.
 - Nếu đủ hàng, tạo đơn hàng, giảm tồn kho, trả về thông tin đơn hàng.
 - Nếu không đủ hàng, trả về lỗi.
- Xem danh sách đơn hàng (List Orders):
 - Người dùng gửi request kèm JWT token.
 - Hệ thống xác thực token, trả về danh sách đơn hàng của user đó.
- Xem chi tiết đơn hàng (Get Order Detail):
 - Người dùng gửi request với id đơn hàng, kèm JWT token.
 - Hệ thống xác thực token, kiểm tra quyền truy cập, trả về chi tiết đơn hàng nếu user là chủ sở hữu.

1.4. Xác thực và phân quyền bằng JWT

- Khi đăng nhập thành công, hệ thống sinh ra một JWT token chứa thông tin user.
- Mọi request đến các API cần bảo vệ đều phải gửi kèm header:
Authorization: Bearer <token>
- API Gateway sẽ kiểm tra tính hợp lệ của token, giải mã để lấy thông tin user, phân quyền truy cập cho các chức năng phù hợp (ví dụ: chỉ admin mới được thêm/sửa/xóa sản phẩm).

1.5. API Gateway làm entry point duy nhất cho client

- Tất cả request từ client đều đi qua API Gateway.

- Gateway xác thực JWT, định tuyến request đến đúng service phía sau (user, product, order).
- Gateway giúp bảo mật, che giấu nội bộ hệ thống, dễ dàng mở rộng, quản lý log, giám sát, rate limiting, v.v.

2. Các tính năng cụ thể

2.1.User Service

- Đăng ký tài khoản mới
- Đăng nhập, sinh JWT
- Lấy thông tin user theo id

2.2.Product Service

- Thêm sản phẩm mới (admin)
- Sửa/xóa sản phẩm (admin)
- Xem danh sách sản phẩm (có phân trang)
- Xem chi tiết sản phẩm
- Giảm tồn kho khi có đơn hàng

2.3.Order Service

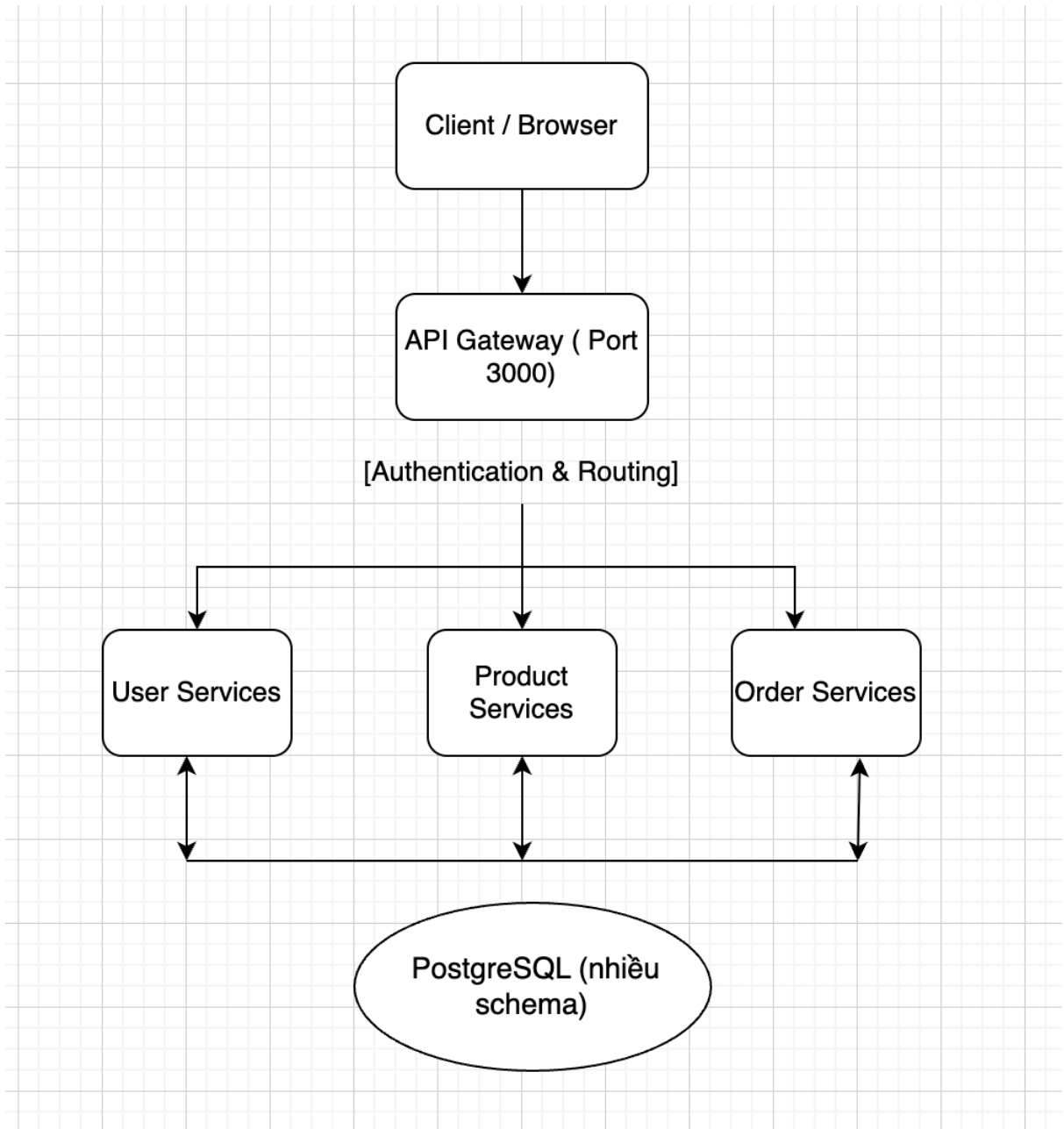
- Tạo đơn hàng mới (cần xác thực)
- Xem danh sách đơn hàng của user
- Xem chi tiết đơn hàng

2.4.API Gateway

- Xác thực JWT cho các route cần bảo vệ
- Định tuyến request đến các service tương ứng
- Forward userId sang order-service qua header

3. Thiết kế phần mềm

3.1. Kiến trúc tổng thể



- API Gateway: Entry point duy nhất, xác thực JWT, định tuyến request.
- User Service: Quản lý user, xác thực, sinh JWT.
- Product Service: CRUD sản phẩm, quản lý tồn kho.
- Order Service: Quản lý đơn hàng, gọi product-service để kiểm tra và giảm tồn kho.
- PostgreSQL: Dùng chung 1 instance, mỗi service 1 schema riêng biệt.
- Redis: (nếu có) dùng cho cache hoặc pub/sub.

3.2. Database Design

Schema: user_schema

- users: id (UUID), email, password, name, created_at, updated_at

Schema: product_schema

- products: id (UUID), name, description, price, stock, image_url, created_at, updated_at

Schema: order_schema

- orders: id (UUID), user_id, total_amount, status, created_at, updated_at
- order_items: id (UUID), order_id, product_id, product_name, quantity, price

3.3. Giao tiếp giữa các service

- API Gateway → User/Product/Order Service: HTTP REST
- Order Service → Product Service: HTTP REST (kiểm tra tồn kho, giảm tồn kho)

3.4. Authentication Flow

1. User đăng ký/đăng nhập → nhận JWT
2. Client gửi JWT qua header Authorization cho các request cần xác thực
3. API Gateway verify JWT, forward request kèm userId cho order-service

4. Demo

4.1. User services

The screenshot shows the 'Register' endpoint in a REST client. The method is POST and the URL is http://localhost:3000/api/auth/register. The request body is a JSON object with the following fields: email, password, and name. The response is a 201 Created status with a JSON object containing email, name, id, createdAt, and updatedAt fields.

Microservices / User services / Register

POST http://localhost:3000/api/auth/register

Send

Docs Params Auth Headers (10) Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "email": "pink@gmail.com",
3   "password": "123456",
4   "name": "Pink"
5 }
```

Body 201 Created • 203 ms • 435 B | Save Response

{ JSON Preview Visualize

```
1 {
2   "email": "pink@gmail.com",
3   "name": "Pink",
4   "id": "1626a2b4-427e-4000-aa0f-08c29685539b",
5   "createdAt": "2025-12-26T11:38:39.539Z",
6   "updatedAt": "2025-12-26T11:38:39.539Z"
7 }
```

The screenshot shows the 'Login' endpoint in a REST client. The method is POST and the URL is http://localhost:3000/api/auth/login. The request body is a JSON object with the following fields: email and password. The response is a 201 Created status with a JSON object containing an access_token field.

Microservices / User services / Login

POST http://localhost:3000/api/auth/login

Send

Docs Params Auth Headers (10) Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "email": "pink@gmail.com",
3   "password": "123456"
4 }
```

Body 201 Created • 198 ms • 513 B | Save Response

{ JSON Preview Visualize

```
1 {
2   "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxNjY2YTJiNC00Mjd1LTQwMDAtYWVwZWZiOjE3NjcwMTIyNzB9.0fvNt6sHlkkC91VTkqaYJsIp0YjVMgkwrB8GhJTJLw"
3 }
```

Microservices / User services / **Get user profile** Save Share

GET http://localhost:3000/api/users/1626a2b4-427e-4000-aa0f-08c29685539b Send

Docs Params Auth **Headers (9)** Body Scripts Settings Cookies

	Key	Value	Descri...	Bulk Edit	Presets
☐	x-user-id	1626a2b4-427e-4000-aa0f-08c296...			
☒	Authorization	Bearer {{access_token}}			
	Key	Value	Description		

Body 200 OK 78 ms 430 B Save Response

JSON Preview Visualize

```
1 {
2   "id": "1626a2b4-427e-4000-aa0f-08c29685539b",
3   "email": "pink@gmail.com",
4   "name": "Pink",
5   "createdAt": "2025-12-26T11:38:39.539Z",
6   "updatedAt": "2025-12-26T11:38:39.539Z"
7 }
```

4.2.Product service

Microservices / Product services / **List products** Save Share

GET http://localhost:3000/api/products Send

Docs **Params** Auth Headers (7) Body Scripts Settings Cookies

Query Params

	Key	Value	Description	Bulk Edit
--	-----	-------	-------------	-----------

Body 200 OK 36 ms 577 B Save Response

JSON Preview Visualize

```
1 {
2   "data": [
3     {
4       "id": "28908e95-4eb2-427a-b42c-779b869be587",
5       "name": "Book 2",
6       "description": "description for book 1",
7       "price": "2599.99",
8       "stock": 10,
9       "imageUrl": "https://example.com/macbook.jpg",
10      "createdAt": "2025-12-28T12:38:48.812Z",
11      "updatedAt": "2025-12-28T12:55:12.397Z"
12    }
13  ],
14  "total": 1,
15  "page": 1,
16  "limit": 10,
17  "totalPages": 1
18 }
```

Microservices / Product services / **Create products** Save Share

POST http://localhost:3000/api/products Send

Docs Params Auth Headers (11) **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Sách BỐ Già",
3   "description": "description for sách BỐ Già",
4   "price": 30000,
5   "stock": 5,
6   "imageUrl": "https://example.com/macbook.jpg"
7 }
```

Body 201 Created • 75 ms • 539 B | Save Response

{} JSON Preview Visualize

```
1 {
2   "name": "Sách BỐ Già",
3   "description": "description for sách BỐ Già",
4   "price": 30000,
5   "stock": 5,
6   "imageUrl": "https://example.com/macbook.jpg",
7   "id": "1b7d257a-d6a6-421b-b0c7-64084627b23e",
8   "createdAt": "2025-12-28T12:56:17.426Z",
9   "updatedAt": "2025-12-28T12:56:17.426Z"
10 }
```

Microservices / Product services / **Update products** Save Share

PUT http://localhost:3000/api/products/{{product_id}} Send

Docs Params Auth Headers (11) **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Book 2",
3   "price": 2599.99
4 }
```

Body 200 OK • 70 ms • 518 B | Save Response

{} JSON Preview Visualize

```
1 {
2   "id": "28908e95-4eb2-427a-b42c-779b869be587",
3   "name": "Book 2",
4   "description": "description for book 1",
5   "price": 2599.99,
6   "stock": 10,
7   "imageUrl": "https://example.com/macbook.jpg",
8   "createdAt": "2025-12-28T12:38:48.812Z",
9   "updatedAt": "2025-12-28T12:55:12.397Z"
10 }
```

Microservices / Product services / Delete products

DELETE

http://localhost:3000/api/products/ {{product_id}}

Send

Docs

Params

Auth

Headers (8)

Body

Scripts

Settings

Cookies

Headers

7 hidden

	Key	Value	Descrip...	...	Bulk Edit	Presets
☒	Authorization	Bearer {{access_token}}				
	Key	Value	Description			

Body

200 OK

46 ms

267 B

Save Response

JSON

Preview

Visualize

1

Microservices / Product services / Reduce stocks

POST

http://localhost:3000/api/products/ {{product_id}} /reduce-stock

Send

Docs

Params

Auth

Headers (11)

Body

Scripts

Settings

Cookies

Headers

9 hidden

	Key	Value	Descrip...	...	Bulk Edit	Presets
☒	Content-Type	application/json				
☒	Authorization	Bearer {{access_token}}				
	Key	Value	Description			

Body

201 Created

47 ms

524 B

Save Response

JSON

Preview

Visualize

1

{

2

"id": "28908e95-4eb2-427a-b42c-779b869be587",

3

"name": "Book 2",

4

"description": "description for book 1",

5

"price": "2599.99",

6

"stock": 4,

7

"imageUrl": "https://example.com/macbook.jpg",

8

"createdAt": "2025-12-28T12:38:48.812Z",

9

"updatedAt": "2025-12-28T13:03:39.913Z"

10

}

Microservices / Product services / Check stocks

POST http://localhost:3000/api/products/ {{product_id}} /check-stock

Send

Docs Params Auth Headers (11) Body Scripts Settings Cookies

Headers 9 hidden

	Key	Value	Descrip...	Bulk Edit	Presets
☒	Content-Type	application/json			
☒	Authorization	Bearer {{access_token}}			
	Key	Value	Description		

Body 201 Created 20 ms 290 B Save Response

JSON Preview Visualize

```
1 {
2   "available": true
3 }
```

4.3.Order Services

Microservices / Orders services / create orders

POST http://localhost:3000/api/orders

Send

Docs Params Authorization Headers (11) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "items": [
3     {
4       "productId": {{product_id}},
5       "quantity": 2
6     }
7   ]
8 }
```

Body Cookies Headers (8) Test Results 201 Created 290 ms 700 B Save Response

JSON Preview Visualize

```
1 {
2   "id": "413bc088-0c8a-4269-8279-199d0b68cf85",
3   "userId": "1626a2b4-427e-4000-aa0f-08c29685539b",
4   "totalAmount": "5199.98",
5   "status": "confirmed",
6   "createdAt": "2025-12-28T13:03:28.116Z",
7   "updatedAt": "2025-12-28T13:03:28.190Z",
8   "items": [
9     {
10      "id": "3b4f1d49-5c2d-493c-a666-0a1c49aa8fbc",
11      "orderId": "413bc088-0c8a-4269-8279-199d0b68cf85",
12      "productId": "28908e95-4eb2-427a-b42c-779b869be587",
13      "productName": "Book 2",
14      "quantity": 2,
15      "price": "2599.99"
16    }
17  ]
18 }
```


5. Hướng Dẫn Chạy Demo

5.1. Yêu cầu hệ thống

- Docker & Docker Compose
- Node.js ≥ 18 (nếu muốn chạy dev mode)
- Git

5.2. Các bước chạy demo

Clone project

```
git clone <repository-url>
```

```
cd microservices-demo-nestjs
```

Build và start toàn bộ hệ thống

```
docker-compose up --build
```

(Lần đầu cần tạo schema DB)

```
./setup-db-schema.sh
```

```
docker compose restart
```

Xem log

```
docker-compose logs -f
```

Dừng hệ thống

```
docker-compose down
```

5.3. Thông tin kết nối

- API Gateway: <http://localhost:3000>
- User Service: <http://localhost:3001>
- Product Service: <http://localhost:3002>
- Order Service: <http://localhost:3000>
- PostgreSQL: localhost:5432 (user: postgres, pass: postgres123, db: ecommerce_db)

6. Source Code

<https://github.com/huongpink2711/Nh-m-9---Microservices---Demo.git>